



兰州工业学院

实 验 报 告

(2023 —2024 学年第 1 学期)

课程名称 面向对象程序设计

专业班级 专升本电信 23

学 号 ZB2305010116

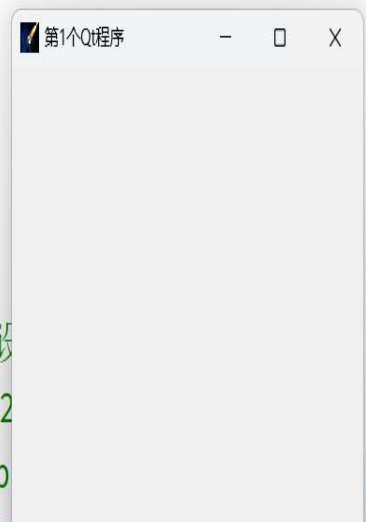
姓 名 姜 勇

实验项目	实验三 Qt 的安装和使用		
实验时间	2023 年 10 月 30 日	实验地点	信息与通信工程专业实验室 (DSP)
一、实验目的 1. 熟悉 Qt Creator 集成开发环境； 2. 熟悉简单的 Qt 图形用户界面程序的编写； 3. 掌握发布程序、设置程序图标等常用操作。 二、实验环境 硬件：计算机一台； 软件：VC++ 6.0 或 Code::Blocks。 三、实验内容 1. 安装和配置 Qt 开发系统。 在 Qt 官网上找到各种版本的 Qt，根据自己的需求选择一个版本下载安装包。 双击下载好的安装包，在安装向导中选择自定义安装选项，选择自己需要的插件比如说 Mingw，score 等，以便更精细地控制安装配置。 配置 Qt Creator。启动 Qt Creator，打开“工具”菜单下的“选项”对话框，设置 Qt 版本、编译器、Qt 安装路径等。 2. 分别使用手工编码方式和 Qt 设计器方式，创建一个 QMainWindow 类型的窗口（主窗口类型，注意修改包含的库文件）。 （1）将主窗口的标题设置为“第 1 个 Qt 程序”（使用成员函数 <code>setWindowTitle</code>），将主窗口的大小设置为 300 像素宽、300 像素高（使用成员函数 <code>resize</code>）； <pre>setWindowTitle("第 1 个 Qt 程序"); resize(300, 300);</pre>			

```

3  Widget::Widget(QWidget *parent)
4      : QWidget(parent)
5  {
6      setWindowTitle("第1个Qt程序");
7      resize(300, 300);
8      // label1.setParent(this);
9      // label2.setParent(this);
10     // label1.setText("Hello Qt");//设
11     // label1.setFont(QFont("宋体", 12
12     // label1.setStyleSheet("color: b

```



(2) 在窗口中添加两个 label 标签，显示结果如图 1 所示，自定义显示内容及格式。

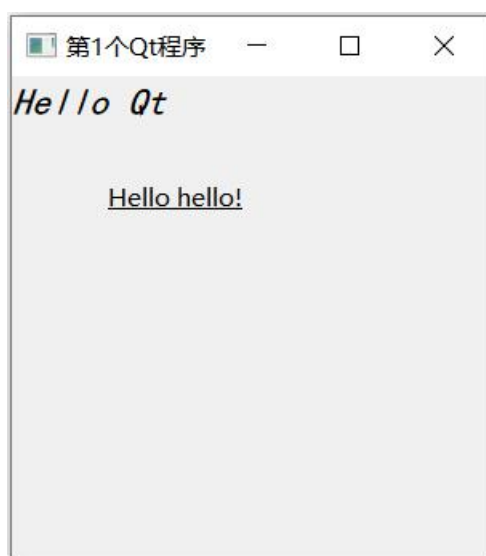


图 1 页面显示效果

Widget.h 文件

```

#ifndef WIDGET_H
#define WIDGET_H
#include <QLabel>
#include <QWidget>
class Widget : public QWidget
{
    Q_OBJECT
public:

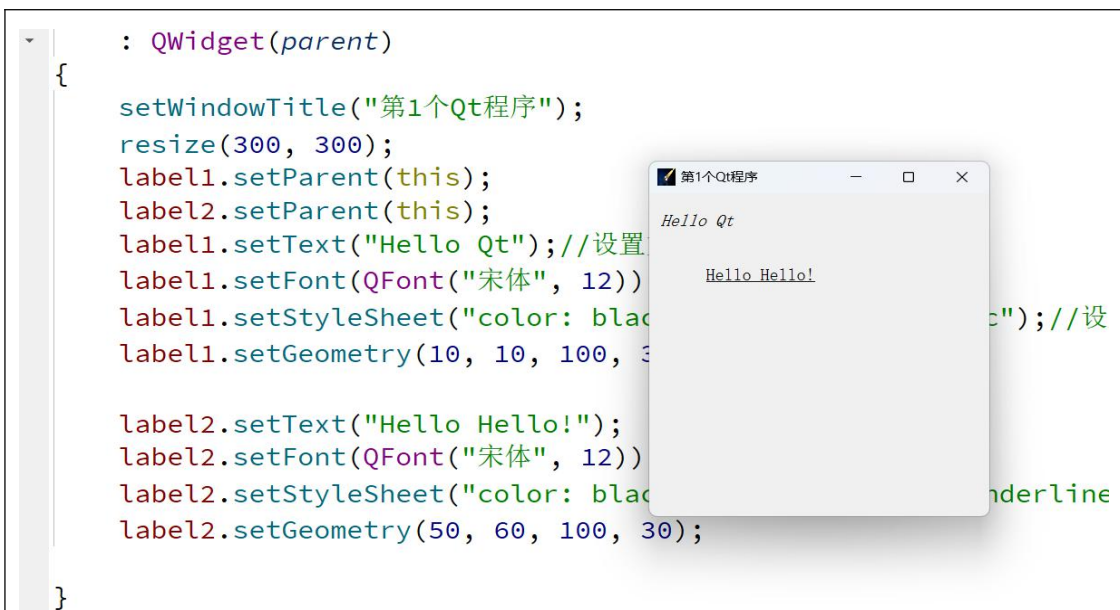
```

```

Widget(QWidget *parent = nullptr);
~Widget();
QLabel label1, label2;
};
#endif // WIDGET_H
Widget.cpp 文件
#include "widget.h"
Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    setWindowTitle("第 1 个 Qt 程序");
    resize(300, 300);
    label1.setParent(this);
    label2.setParent(this);
    label1.setText("Hello Qt");//设置文本内容
    label1.setFont(QFont("宋体", 12));//设置字体为宋体 12 号
    label1.setStyleSheet("color: black; font-style: italic");//设置文
本颜色为黑色字体样式为斜体
    label1.setGeometry(10, 10, 100, 30);//设置位置和大小
    label2.setText("Hello Hello!");
    label2.setFont(QFont("宋体", 12));
    label2.setStyleSheet("color:                black;text-decoration:
underline");//字体样式设置为下划线
    label2.setGeometry(50, 60, 100, 30);
}

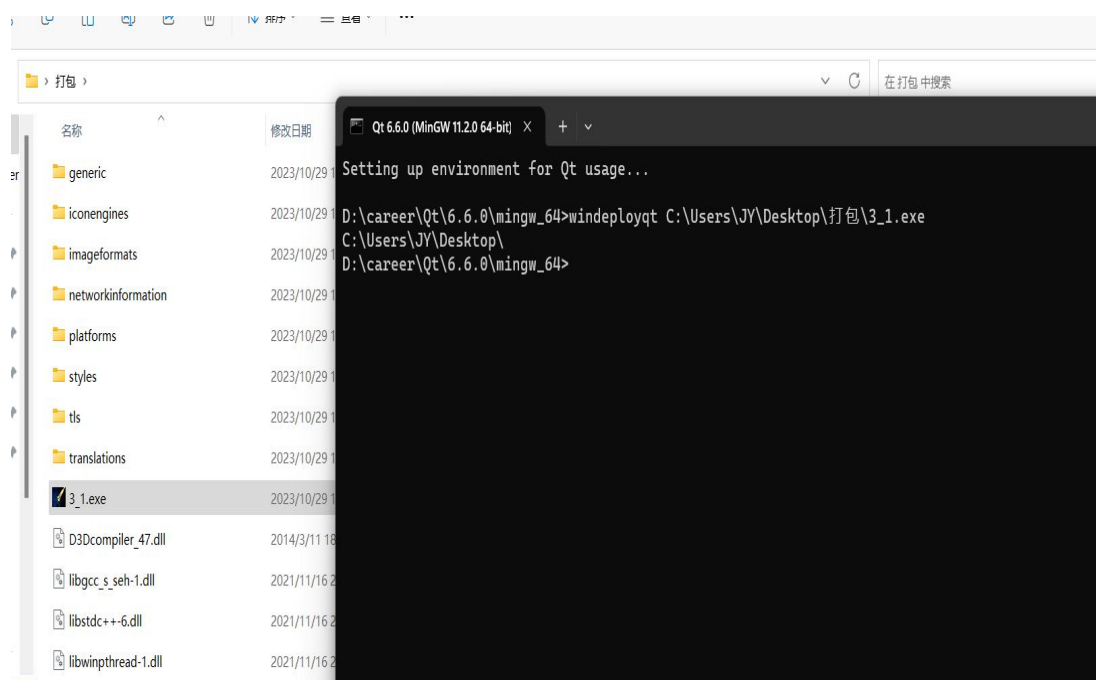
Widget::~Widget()
{
}

```



(3) 在实验内容 2 基础上给应用程序添加图标，并实现程序的发布。

在 .pro 项目文件中添加 RC_ICONS=1. ico



3. 设计一个简单计算器实现基本的四则运算功能。

CalculatorWidget.h 文件

```

#ifndef CALCULATORWIDGET_H
#define CALCULATORWIDGET_H
#include <QWidget>
#include <QVBoxLayout>

```

```

#include <QPushButton>
#include <QLineEdit>
#include <QLabel>
class CalculatorWidget : public QWidget
{
    Q_OBJECT
public:
    explicit CalculatorWidget(QWidget *parent = nullptr);
private slots:
    void add();
    void subtract();
    void multiply();
    void divide();
private:
    QLineEdit input1, input2;
    QLabel resultLabel;
    QPushButton jia, jian, cheng, chu;
    QVBoxLayout layout;
    QHBoxLayout buttonLayout;
};
#endif // CALCULATORWIDGET_H

CalculatorWidget.cpp 文件
#include "calculatorwidget.h"
#include <QMessageBox>
CalculatorWidget::CalculatorWidget(QWidget *parent) : QWidget(parent)
{
    setWindowTitle("简单计算器");
    resize(500, 150);
    layout.setParent(this); //将布局管理器 layout 设置为当前小部件的父

```

级。

```
input1.setParent(this);
input1.resize(500, 20);
input2.setParent(this);
input2.move(0, 50);
input2.resize(500, 20);
resultLabel.setParent(this);
resultLabel.resize(200, 100);
resultLabel.move(0, 60);
jia.setParent(this);
jia.setText("+");
jia.move(0, 125);
jian.setParent(this);
jian.setText("-");
jian.move(140, 125);
cheng.setParent(this);
cheng.setText("*");
cheng.move(280, 125);
chu.setParent(this);
chu.setText("/");
chu.move(420, 125);
```

//将按钮的 clicked() 信号与对应的槽函数进行绑定，使得当对应的按钮被点击时，对应的槽函数将会被调用。

```
connect(&jia, SIGNAL(clicked()), this, SLOT(add()));
connect(&jian, SIGNAL(clicked()), this, SLOT(subtract()));
connect(&cheng, SIGNAL(clicked()), this, SLOT(multiply()));
connect(&chu, SIGNAL(clicked()), this, SLOT(divide()));
```

//将文本输入框、按钮和结果标签添加到布局管理器中，以便进行界面布局。

```

        layout.addWidget(&input1);
        layout.addWidget(&input2);
        buttonLayout.addWidget(&jia);
        buttonLayout.addWidget(&jian);
        buttonLayout.addWidget(&cheng);
        buttonLayout.addWidget(&chu);
        layout.addLayout(&buttonLayout);
        layout.addWidget(&resultLabel);
    }

void CalculatorWidget::add()
{
    double num1 = input1.text().toDouble();
    double num2 = input2.text().toDouble();
    double result = num1 + num2;
    resultLabel.setText(QString("结果:  %1").arg(result));
}

void CalculatorWidget::subtract()
{
    double num1 = input1.text().toDouble();
    double num2 = input2.text().toDouble();
    double result = num1 - num2;
    resultLabel.setText(QString("结果:  %1").arg(result));
}

void CalculatorWidget::multiply()
{
    double num1 = input1.text().toDouble();
    double num2 = input2.text().toDouble();
    double result = num1 * num2;
    resultLabel.setText(QString("结果:  %1").arg(result));
}

```



```

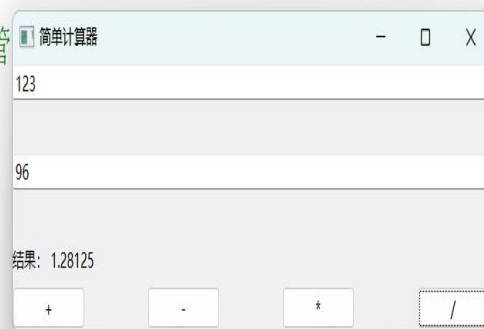
}

void CalculatorWidget::divide()
{
    double num1 = input1.text().toDouble();
    double num2 = input2.text().toDouble();
    if (num2 == 0) {
        QMessageBox::critical(this, "错误", "除数不能为零!");
    } else {
        double result = num1 / num2;
        resultLabel.setText(QString("结果: %1").arg(result));
    }
}
}

CalculatorWidget::CalculatorWidget(QWidget *parent) : QWidget(parent)
{
    setWindowTitle("简单计算器");
    resize(500,150);
    layout.setParent(this); //将布局管
    input1.setParent(this);
    input1.resize(500,20);

    input2.setParent(this);
    input2.move(0,50);
    input2.resize(500,20);

```



四、实验总结

通过本次实验，懂得了在 Qt Creator 中，可以使用项目向导创建项目，选择合适的项目模板，配置项目设置，添加源文件、资源文件、头文件和库等。可以通过添加更多文件和运行配置来修改或扩展项目。知道了一些 qt 框架的控

件，如果要来使用这些控件的话，都必须加入头文件，不然的话无法使用，这些控件都可以用一些方法来设置样式，来实现不同的效果，让用户来自由实现页面布局。还懂了 qt 的信号和槽，Qt 的信号和槽机制为处理用户交互提供了方便的方式，可以使用信号函数的参数来获取用户的输入，并将用户输入传递给槽函数进行处理，也可以使用信号和槽进行类之间的通信。可以根据实际需求创建自定义的信号和槽函数，将其绑定到需要的控件上。

学会了如何打包发布项目，在发布应用程序时，可以生成可执行文件和动态链接库，并打包应用程序的依赖项，按照 Qt 所支持的平台和格式进行编译，并进行应用程序的安装。

实验三成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			